

How a Unified Development Toolchain Supports Automated Testing Consistency in a TypeScript Full-Stack Monorepo: A Case Study of Circles

Aldiyar Seylkanov

*Bachelor Degree Student, School of Software Engineering
Astana IT University (AITU)*

Astana, Kazakhstan

seylkanov.aldiyar@gmail.com

(*orcid*)

Alexandr Tyulkov

*Master Degree Student, School of Software Engineering
Astana IT University (AITU)*

Astana, Kazakhstan

widesehl@gmail.com

0009-0009-6422-0559

Abstract—TypeScript monorepos often use different tools for bundling, testing, linting, and formatting across packages. This fragmentation causes the same test code to behave differently depending on which package runs it. This paper examines how adopting a unified development toolchain addresses this inconsistency. The Circles project, a full-stack TypeScript monorepo, is used as a case study. Before unification, each package configured its own test runner and transform pipeline. After adopting Vite+, all packages share a single tool entry point, one module resolver, and one transform pipeline. The paper describes the toolchain architecture, identifies the sources of inconsistency in the fragmented setup, and evaluates the outcome using configuration surface size, test environment uniformity, and CI pipeline structure. Results show that unification eliminates the main categories of cross-package test inconsistency without requiring per-package configuration.

Index Terms—monorepo, TypeScript, unified toolchain, automated testing, Vite, software engineering

I. INTRODUCTION

II. RELATED WORK

Prior work relevant to this study comes from three areas: large shared repositories, automation around build and test pipelines, and language support for maintaining large scripting codebases. The literature explains why large codebases accumulate coordination costs and why automation matters, but it does not directly evaluate testing consistency as an outcome of toolchain unification in TypeScript monorepos.

A. Large Repositories and Shared Tooling

Brooks argued that software engineering difficulty is driven largely by essential complexity rather than by any single accidental deficiency [1]. This distinction remains useful for monorepo tooling. Tooling fragmentation is not the whole software problem, but it is one accidental source of avoidable complexity because teams must reason about multiple build, test, and resolution environments at once.

Potvin and Levenberg described the operational logic of a single shared repository at Google [2]. Their account identifies the main benefits of repository unification, including shared visibility, atomic changes, simplified dependency

management, and large-scale refactoring. At the same time, the paper notes that these benefits depend on substantial supporting infrastructure for builds, tests, and code health. This is directly relevant to the present study. A monorepo alone does not guarantee consistent behavior across packages. Repository centralization creates the need for common tooling, but it does not by itself explain how test environments can be kept uniform in a TypeScript stack.

The literature on monolithic repositories therefore motivates the problem setting but stops short of the question examined here. Existing work shows that large repositories benefit from shared infrastructure, yet it does not analyze whether a unified resolver and transform pipeline reduce cross-package test inconsistency in JavaScript or TypeScript projects.

B. Maintainability in Typed Scripting Languages

Research on typed scripting languages provides a second useful background. Tobin-Hochstadt and Felleisen observed that scripts often grow into larger systems and become harder to maintain because design information must be rediscovered during change [3]. Their Typed Scheme work is relevant because it frames maintenance problems in ecosystems that begin with flexible scripting practices and later require stronger structure.

Bierman, Abadi, and Torgersen made a similar point for TypeScript, describing it as an extension intended to support large-scale JavaScript applications while preserving common JavaScript idioms [4]. Their analysis emphasizes pragmatic compatibility over full static soundness, reflecting the reality that large JavaScript systems evolve incrementally rather than through complete rewrites. For the present paper, this literature matters because full-stack TypeScript monorepos often combine browser code, server code, tests, and shared libraries that must remain compatible with established JavaScript tooling patterns.

These language papers do not study build or test toolchains directly. However, they establish why TypeScript projects tend to value gradual adoption, compatibility, and low-friction maintenance. Those same constraints make fragmented per-package test infrastructure costly. A tool-

chain that requires separate configuration strategies across packages works against the incremental maintenance model described in this literature.

C. Continuous Integration and Testing Automation

The third relevant area is continuous integration. Hilton et al. define CI systems as automation for compilation, building, and testing, and show that CI usage became widespread in open-source practice because it supports faster and more frequent integration [5]. This line of work demonstrates the value of automating verification, but it treats the CI system primarily as an orchestration layer around project commands.

That distinction is important. CI can run tests consistently only if the underlying local test commands already behave consistently. A pipeline may be fully automated while still invoking different runners, transform steps, or module resolution rules in different packages. The CI literature therefore explains why automated testing infrastructure matters, but it does not isolate the effect of a unified local toolchain on test behavior inside a monorepo.

In summary, prior work explains the pressures created by large shared repositories, the maintenance challenges of large scripting codebases, and the organizational value of build-and-test automation. The reviewed sources do not address the narrower question investigated in this paper: whether sharing one CLI entry point, one resolver, and one transform pipeline across packages improves testing consistency in a TypeScript monorepo. That gap motivates the Circles case study.

III. SYSTEM DESCRIPTION

IV. EVALUATION

V. DISCUSSION

VI. CONCLUSION

ACKNOWLEDGMENT

REFERENCES

- [1] J. Brooks Frederick P., “No Silver Bullet: Essence and Accident in Software Engineering,” technical report TR86–20, 1986.
- [2] R. Potvin and J. Levenberg, “Why Google Stores Billions of Lines of Code in a Single Repository,” *Communications of the ACM*, vol. 59, no. 7, pp. 78–87, 2016, doi: 10.1145/2854146.
- [3] S. Tobin-Hochstadt and M. Felleisen, “The Design and Implementation of Typed Scheme,” in *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ACM, 2008, pp. 395–406.
- [4] G. Bierman, M. Abadi, and M. Torgersen, “Understanding TypeScript,” *Lecture Notes in Computer Science*, vol. 8586, pp. 257–281, 2014.
- [5] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, “Usage, Costs, and Benefits of Continuous Integration in Open-Source Projects,” in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, ACM, 2016, pp. 426–437.